Applied Project 2

Reinforcement Learning from Human Feedback

Misenta Loïc (330593)[1] **Gatti Martina (341013)**[1] **Gotti Bryan (325403)**[1]

[1]EPFL, Lausanne, Switzerland

loic.misenta@epfl.ch martina.gatti@epfl.ch bryan.gotti@epfl.ch

Abstract

We study preference-based reinforcement learning in one discrete and one continuous control environment: CartPole-v1 and Pendulum-v1. Starting from two policies of contrasting quality trained with Proximal Policy Optimization (PPO), we construct preference datasets of varying sizes ($K \in \{500, 1000, 5000, 10000\}$) using Bradley–Terry labelling, and use them to fine-tune policies via two RLHF algorithms: PPO-RLHF, which first learns a reward model from preferences before running PPO, and Direct Preference Optimization (DPO), which optimises the policy directly from preference pairs. We evaluate both methods using environment rewards across 20 random seeds and analyse how dataset size affects sample efficiency and final performance in each environment.

1 Introduction

Reinforcement learning from human feedback (RLHF) has emerged as a powerful technique for aligning agent behaviour with human intent, most prominently in the training of large language models [1, 2]. Rather than relying on a hand-crafted reward signal, RLHF elicits feedback in the form of pairwise preferences between agent trajectories and uses this signal to guide policy learning.

In this project, we apply preference-based learning to simple control environments, CartPole-v1, a discrete-action balancing task with dense rewards, and Pendulum-v1, a continuous-action torque-control task with shaped but challenging rewards, and compare two fundamentally different RLHF algorithms as the amount of preference data varies. PPO-RLHF [3, 4] follows a classical two-stage pipeline. It first trains a reward model on the preference data, then runs PPO using that learned reward in place of the true environment reward. DPO [5] skips the reward model entirely. It can be shown that the optimal policy under a KL-penalised objective has a closed-form relationship with the reward function, which means the reward can be expressed directly in terms of policy log-probabilities, allowing DPO to optimise the policy in a single stage from preference pairs alone.

For each environment, we train a strong policy π_1 and a mediocre policy π_2 using PPO, generate preference datasets at four dataset sizes $K \in \{500, 1000, 5000, 10000\}$, and evaluate both RLHF methods against a standard PPO baseline using true environment rewards over 20 independent seeds. Our goal is to understand how each algorithm's performance scales with the amount of preference data, and whether this scaling behaviour differs across algorithms and environments.

2 Background and Related Work

The two RLHF algorithms we compare both build on PPO as their underlying policy optimiser.

2.1 Proximal Policy Optimization

PPO [3] is an on-policy actor-critic algorithm that constrains policy updates using a clipped surrogate objective:

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t\Big[\min\Big(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\varepsilon, 1+\varepsilon)\hat{A}_t\Big)\Big] \tag{1}$$

where $r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\text{old}}(a_t \mid s_t)}$ is the probability ratio and $\hat{A}_t$ is an estimate of the advantage function. The clipping mechanism prevents excessively large updates, providing stable training without the need for a hard KL constraint. PPO serves as both the training algorithm for π_1 and π_2 in Section 3.2, and as the policy optimiser inside PPO-RLHF in Section 3.4.

2.2 PPO-RLHF

PPO is agnostic to the source of its reward signal. It simply consumes a scalar at each step and optimises accordingly. This makes it straightforward to extend to the RLHF setting. If we can learn a function that predicts how good a trajectory is from preference data, we can plug it directly into PPO without changing anything else about the algorithm. This is the idea behind PPO-RLHF [2].

Concretely, a reward model R_φ is trained on a dataset of K preference pairs $\{(\tau_i^+, \tau_i^-)\}_{i=1}^K$, where τ^+ is preferred over τ^-, by minimising the Bradley–Terry cross-entropy loss:

$$\mathcal{L}_{\text{RM}}(\varphi) = -\frac{1}{K}\sum_{i=1}^{K} \log\sigma\Big(R_\varphi(\tau_i^+) - R_\varphi(\tau_i^-)\Big) \tag{2}$$

where $R_\varphi(\tau) = \sum_{t=0}^{T} r_\varphi(s_t, a_t)$ sums per-step scores along the trajectory, and σ is the logistic function. Once training converges, the reward model weights are frozen, and standard PPO is run using r_φ as a drop-in replacement for the true environment reward:

$$\max_\theta \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T} r_\varphi(s_t, a_t)\right] \tag{3}$$

This two-stage approach is straightforward but introduces a potential bottleneck. If K is small, the reward model may be inaccurate, and any errors are then amplified during policy optimisation.

2.3 Direct Preference Optimization

DPO [5] asks whether the reward modelling step can be skipped entirely. The key insight is that under a KL-regularised objective, the optimal policy has a closed-form relationship with the reward function. This means the reward can be reparametrised in terms of policy log-ratios, and the Bradley–Terry loss can be rewritten as a loss directly on the policy π_θ, without ever explicitly representing a reward:

$$\mathcal{L}_{\text{DPO}}(\theta) = -\frac{1}{K}\sum_{i=1}^{K} \log\sigma\left(\beta\log\frac{\pi_\theta(\tau_i^+)}{\pi_{\text{ref}}(\tau_i^+)} - \beta\log\frac{\pi_\theta(\tau_i^-)}{\pi_{\text{ref}}(\tau_i^-)}\right) \tag{4}$$

where π_{ref} is a fixed reference policy that anchors the updates, and $\beta > 0$ controls how far π_θ is allowed to diverge from π_{ref}. In our setting, π_{ref} is set to π_2, the mediocre policy used to generate the preference data. Unlike PPO-RLHF, DPO is fully offline. Once the preference dataset is collected, no further environment interaction is needed.

3 Methods

3.1 Libraries and frameworks

All reinforcement learning experiments are implemented using the Stable-Baselines3 library [6] in combination with the Gymnasium environment API [7], which builds upon the classic OpenAI Gym framework [8]. Stable-Baselines3 provides a standardized and well-tested implementation of PPO, including logging, vectorized environments, and evaluation utilities, while Gymnasium is used as the interface for all control environments (CartPole-v1, Pendulum-v1, and others in the initial sweep). This setup ensures reproducibility across seeds and allows consistent evaluation of policies during training using the built-in evaluation callbacks. Unless otherwise specified, we use the default PPO implementation from Stable-Baselines3, which corresponds to the clipped surrogate objective.

3.2 Training Two Policies

The first step is to obtain two policies of contrasting quality for each environment: a strong policy π_1 that performs near-optimally, and a mediocre policy π_2 that performs significantly worse. These two policies serve a specific purpose in the next step: they are used to generate preference pairs, where trajectories from π_1 are expected to be preferred over trajectories from π_2. The quality of the entire downstream pipeline therefore depends directly on having a clear performance gap between these two policies.

For π_1, the requirement is straightforward. We train the model until convergence, reaching near-maximal mean reward.

For π_2, we aim to capture a checkpoint during the learning phase that has achieved a moderate level of competence but is still clearly inferior to π_1.

3.2.1 Finding the Right Training Stage for π_2

To locate this intermediate policy, we monitor the mean reward throughout the training process. We evaluate the policy over 20 environment seeds at regular intervals to track its progress. Our target for π_2 is a training stage where the mean reward has reached approximately half the optimum. This ensures the policy has moved past completely random behavior but remains distinctly worse than the converged π_1.

3.2.2 Environment Selection

Before settling on CartPole-v1 and Pendulum-v1, we conducted a sweep across several candidate environments (MountainCar-v0, MountainCarContinuous-v0, Acrobot-v1, LunarLander-v3). Not all environments were suitable for our experimental setup: the preference labelling procedure requires PPO to learn reliably, and π_2 to be stably obtainable. MountainCar variants were excluded because PPO failed to produce any useful learning, either due to sparse rewards or convergence to a degenerate policy. Acrobot-v1 and LunarLander-v3 were excluded because their learning transitions made it difficult to reliably pinpoint and extract a stable intermediate policy.

CartPole-v1 and Pendulum-v1 were the two environments that cleanly satisfied all requirements. They were also selected to provide complementary coverage. CartPole-v1 has a discrete action space and converges in tens of thousands of steps, while Pendulum-v1 has a continuous action space and requires an order of magnitude more training. This contrast allows us to assess whether the RLHF algorithms behave consistently across fundamentally different control settings. Both environments are also computationally lightweight, which was a practical constraint given the available hardware.

3.2.3 CartPole-v1

CartPole-v1 has a maximum episode reward of 500. We target $\pi_1 \approx 490$ and $\pi_2 \approx 250$. We trained using the default hyperparameters for PPO, meaning `n_steps` was set to 256 and the learning rate was 10^{-4}.

3.2.4 Pendulum-v1

Pendulum-v1 is a continuous control environment with reward range approximately $[-1600, -100]$. We target $\pi_1 \approx -200$ and $\pi_2 \approx -450$. The default PPO hyperparameters used for CartPole-v1 failed to produce any learning in this environment, requiring a dedicated hyperparameter search before suitable policies could be obtained. The key adjustments were increasing `n_steps` from 256 to 2048 and the learning rate from 10^{-4} to 3×10^{-4}, which gives PPO longer rollouts and a stronger gradient signal to navigate this denser continuous control setting.

3.3 Generating Preference Datasets

To train the reward model and perform direct preference optimization, we construct preference datasets using the previously saved policies π_1 and π_2. We generate four dataset sizes, $K \in \{500, 1000, 5000, 10000\}$, to evaluate how scaling the volume of feedback impacts the downstream performance of both PPO-RLHF and DPO. The generated datasets are saved as serialized files containing the complete state-action-reward histories alongside the assigned binary preference labels.

The generation procedure consists of three core phases: trajectory rollout, stochastic preference labeling, and variance reduction through common random numbers.

3.3.1 Trajectory Rollout

For each pair, we collect a trajectory τ_1 from the strong policy π_1 and a trajectory τ_2 from the mediocre policy π_2. Rollouts are performed deterministically by taking the most likely action from the policy. This allows to guarantee a similar reward value for each trajectory generated by a given policy. Each transition is recorded as a tuple (s_t, a_t, r_t), and we compute the true cumulative trajectory reward $R_i = \sum_t r_t$.

3.3.2 Bradley–Terry Preference Modeling and Numerical Stability

Feedback is simulated using the Bradley–Terry (BT) choice model [2]. The probability that trajectory τ_1 is preferred over τ_2 is modeled as a softmax over their cumulative ground-truth rewards:

$$P(\tau_1 \succ \tau_2) = \frac{\exp(R_1)}{\exp(R_1) + \exp(R_2)} \tag{5}$$

In environments with wide reward ranges, such as Pendulum-v1 (where cumulative rewards can be highly negative, e.g., -1600), directly calculating exponentials of these magnitudes leads to numerical underflow. To prevent computational instability, we implement a stable log-sum-exp subtraction trick. By shifting the values by $C = \max(R_1, R_2)$, the choice probability is computed as:

$$P(\tau_1 \succ \tau_2) = \frac{\exp(R_1 - C)}{\exp(R_1 - C) + \exp(R_2 - C)} \tag{6}$$

Using these probabilities, a binary label $y \in \{0, 1\}$ is sampled, where $y = 0$ designates a preference for τ_1 and $y = 1$ designates a preference for τ_2.

3.3.3 Variance Reduction via Common Random Numbers

Because control environments feature randomized initial state distributions (such as varying starting angles in CartPole or Pendulum), comparing trajectories initiated from different starting states introduces significant confounding variance. A policy might look poor simply because it was initialized in an exceptionally difficult starting configuration.

To isolate policy capability from environmental stochasticity, we employ the method of Common Random Numbers (CRN) [9]. For each paired comparison, we sample a shared environment seed $s_e \sim \mathrm{Uniform}(0, 10^6)$ and use it to initialize the environment reset for both τ_1 and τ_2. Consequently, both policies start from the exact same initial state, ensuring that the resulting preference label strictly reflects the behavioral differences between π_1 and π_2.

3.4 RLHF Algorithms

In the final phase of our pipeline, we utilize the generated preference datasets to fine-tune our agents. To ensure a controlled and rigorous comparison, both DPO and PPO-RLHF are initialized from the same starting policy checkpoint: the mediocre policy π_2. This choice isolates the comparative performance of the two algorithms, as both begin learning from the exact same behavioral baseline.

3.4.1 Reward Model

Before running reinforcement learning in the PPO-RLHF pipeline, we must first construct a proxy reward function from our pairwise preference dataset.

The reward model $R_\varphi(s, a)$ is parameterized as a multi-layer perceptron (MLP) mapping a state-action pair to a scalar value. The state representation s is fed directly as a vector to the model. The action representation a is environment-dependent:

- **Discrete Actions (CartPole-v1):** Actions are converted to one-hot vectors to prevent the network from assuming ordinal relationships between discrete choices.
- **Continuous Actions (Pendulum-v1):** Actions are passed directly as a continuous vector.

The network architecture consists of an input layer of dimension $\dim(s) + \dim(a)$, followed by two hidden layers with Rectified Linear Unit (ReLU) activations, and a final linear layer outputting a single scalar reward:

$$R_\varphi(s, a) = \mathrm{Linear}(\mathrm{ReLU}(\mathrm{Linear}(\mathrm{ReLU}(\mathrm{Linear}([s, a])))))) \tag{7}$$

The predicted return of a trajectory is the sum of its individual step rewards: $R_\varphi(\tau) = \sum_{t=0}^{T} R_\varphi(s_t, a_t)$.

The model is trained by minimizing the Bradley–Terry cross-entropy loss. For every preference pair in the dataset, the data is preprocessed such that τ^+ represents the preferred trajectory and τ^- represents the rejected trajectory. The base loss function is defined as:

$$\mathcal{L}_{\mathrm{RM}}(\varphi) = -\frac{1}{B} \sum_{i=1}^{B} \log \sigma\Big(R_\varphi(\tau_i^+) - R_\varphi(\tau_i^-)\Big) \tag{8}$$

where $B = 16$ is the batch size.

Because severe overfitting was observed during training (particularly on Pendulum-v1), we significantly augmented both the model capacity and the optimization pipeline:

- **Dynamic network capacity:** To restrict the model's ability to memorize smaller datasets while maintaining sufficient capacity for larger ones, the number of units in the two hidden layers is set to 64 if the dataset size $K \leq 1000$, and 256 otherwise.
- **Train/val split and early stopping:** We split each preference dataset using an 80-20 train-validation ratio. Rather than training for a fixed number of epochs, we use the validation loss to trigger early stopping and restore the best-performing model checkpoint.
- **Environment-specific regularization:** We optimize the model using the Adam optimizer with a base learning rate of 10^{-3}, applying targeted regularization based on the environment:
    - **Pendulum-v1:** To further prevent memorization of continuous state-action spaces, we apply a weight decay of 10^{-3} and add a variance penalty ($\lambda_{\text{reg}} = 0.01$) to the loss function.
    - **CartPole-v1:** These regularizations are intentionally disabled. Because CartPole yields a constant $+1$ reward per step, a variance penalty actively fights the cross-entropy objective and causes model collapse. Furthermore, the CartPole reward model is fragile and requires every bit of signal without weight decay interference.

To limit damage and ensure stability when passing the trained reward model to the PPO algorithm, we apply two post-training adjustments:

- **Reward centering:** After the reward model is trained, we roll out the current policy π_t for 50 episodes and adjust the final linear layer's bias so that the outputs are centered around zero.
- **Clipping:** We restrict extreme reward values by clipping the output of the reward model during training of PPO.

3.4.2 PPO-RLHF

For PPO-RLHF, policy updates require active environment interaction. To facilitate this, we construct a custom environment wrapper that intercepts transitions. During training, the true environment reward r_t is discarded and replaced with the scalar prediction generated by the frozen, pre-trained reward model:

$$\hat{r}_t = R_\varphi(s_t, a_t) \tag{9}$$

Standard PPO is then executed on this wrapped environment. The policy is optimized using Adam with a learning rate of 3×10^{-4} over a total of $100,000$ timesteps for CartPole-v1 environment and $1,000,000$ for Pendulum-v1 environment. To maintain stable online rollouts, we set the rollout length `n_steps` to 1024, the batch size to 64, and the discount factor γ to 0.99.

3.4.3 DPO

Unlike PPO-RLHF, DPO is completely offline and bypasses reward modeling entirely. Training is conducted by directly optimizing the policy parameters θ on the static preference dataset.

For each preference pair (τ^+, τ^-), we compute the cumulative action log-probabilities under both the active policy π_θ and the frozen reference policy π_{ref} (parameterized by π_2):

$$\log \pi(\tau) = \sum_{t=0}^{T} \log \pi(a_t \mid s_t) \tag{10}$$

The objective is optimized by minimizing the DPO loss:

$$\mathcal{L}_{\text{DPO}}(\theta) = -\frac{1}{B}\sum_{i=1}^{B} \log \sigma \left(\beta \log \left(\frac{\pi_\theta(\tau_i^+)}{\pi_{\text{ref}}(\tau_i^+)} \right) - \beta \log \left(\frac{\pi_\theta(\tau_i^-)}{\pi_{\text{ref}}(\tau_i^-)} \right) \right) \tag{11}$$

where the implicit reward scaling parameter β is set to 0.1. The model is trained using Adam with a learning rate of 1×10^{-4} for 5 epochs and a batch size of 128.

3.4.4 Evaluation

To prevent evaluation bias and address environmental stochasticity, both algorithms are evaluated periodically during training. Evaluation is conducted by frozen deterministic rollouts over 20 validation episodes. Crucially, these validation episodes are initialized using 20 seeds, disjoint from both the training seeds and the preference dataset generation seeds, ensuring an unbiased assessment of policy generalization.

4 Results

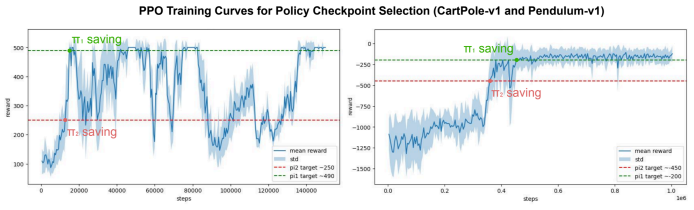

Figure 1: Training curves for PPO on CartPole-v1 (left) and Pendulum-v1 (right). Dashed lines indicate the target mean returns used to select the π_1 (expert, green) and π_2 (mediocre, red) checkpoints.

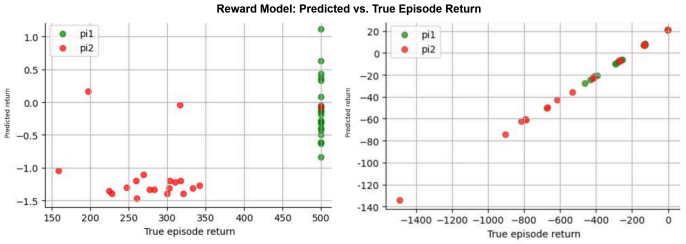

Figure 2: Predicted vs. true episode return for the learned reward model on CartPole-v1 (left) and Pendulum-v1 (right), trained on K=5000 preference pairs. Each point is one episode; green: π_1 (expert), red: π_2 (mediocre).

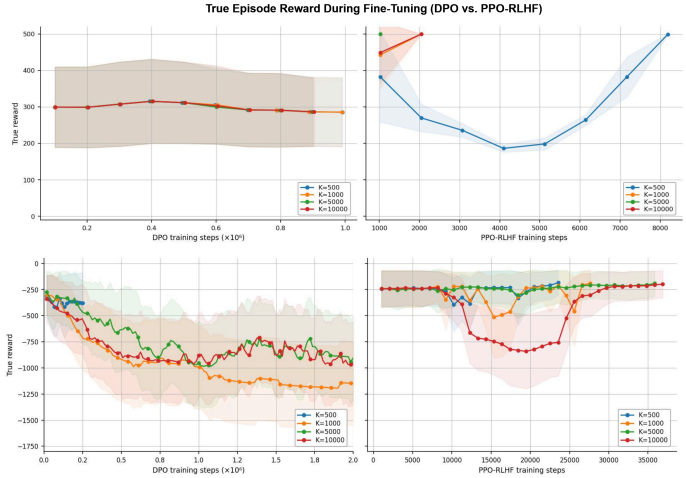

Figure 3: True episode reward during fine-tuning on CartPole-v1 (top) and Pendulum-v1 (bottom), using DPO (left) and PPO-RLHF (right), across preference dataset sizes K. Shaded regions indicate standard deviation across seeds.

Figure 4: Average true episode return at the best checkpoint of fine-tuning as a function of preference dataset size K, on CartPole-v1 (left) and Pendulum-v1 (right). Orange: PPO-RLHF, blue: DPO. Shaded regions indicate standard deviation across seeds. Dashed lines mark the π_1 (expert) and π_2 (mediocre) checkpoint returns for reference.

5 Discussion

5.1 Training of Expert and Mediocre Policies

The distinct performance gaps between π_1 and π_2 established in Figure 1 provided a solid foundation for evaluating the RLHF algorithms, ensuring that the preference datasets contained a clear directional signal. Additionally, the initial training curves highlight the contrasting difficulty of the two environments: while CartPole-v1 converges to an optimal policy relatively quickly, Pendulum-v1 proves to be a much more challenging continuous control task, requiring roughly an order of magnitude more environment steps to reach its maximum reward.

5.2 Reward Model Efficacy

The success of PPO-RLHF depends entirely on the quality of the learned reward model. As shown in Figure 2, the reward models successfully captured the preference signals, but their topographies differed significantly depending on the environment.

In Pendulum-v1, the learned reward acts essentially as a linear transformation of the true environment reward. Because Pendulum provides a dense, continuous reward landscape, the multi-layer perceptron was able to learn a highly faithful proxy of the true objective. This makes the learned reward very useful, as it can completely and seamlessly replace the environment reward during PPO fine-tuning.

In CartPole-v1, the relationship is non-linear and exhibits high variance within the expert and mediocre clusters. This is a direct consequence of omitting variance penalization during training: because the true reward in CartPole is a constant $+1$ per surviving timestep, forcing the model to minimize variance would actively fight the Bradley-Terry objective and cause collapse. Despite this scatter, the reward model remains highly effective for PPO. It establishes a strict boundary separating good trajectories from bad ones. For PPO to learn a successful policy, it does not strictly need a perfectly scaled linear reward; it only needs a consistent gradient direction, which this binary-like separation provides.

5.3 Training Dynamics and Algorithmic Stability

The fine-tuning trajectories in Figure 3 highlight a stark contrast in stability and learning capacity between the two algorithms.

PPO-RLHF consistently solves both environments, but exhibits a distinct "U-curve" dynamic, initially dropping below the performance of π_2 before recovering to expert levels. This initial degradation is likely an exploration phase. When PPO is suddenly evaluated against a new proxy reward landscape rather than the environment's true reward, it must deviate from π_2's pre-trained behavior to explore the state space. Once it discovers the regions that maximize the learned reward—which, as Figure 2 shows, correlate well with the true reward—it rapidly converges to an optimal policy.

Conversely, DPO struggles to improve significantly in CartPole and actively collapses in Pendulum. Extensive manual hyperparameter tuning confirmed that this is a systematic issue rather than a simple configuration error. Lowering the learning rate resulted in slight initial improvements followed by total stagnation, while higher learning rates caused immediate collapse. Similarly, adjusting the KL-regularization parameter β yielded a strict trade-off: a low β caused the policy to drift aimlessly and collapse (likely due to insufficient anchoring

to π_2), while a high β forced the policy to remain too close to the mediocre baseline, preventing meaningful improvement.

We hypothesize several reasons for DPO's poor performance. First, DPO is a strictly offline algorithm. Unlike PPO, which periodically updates its advantage estimates by exploring the environment, DPO relies on a static dataset and a static reference policy. It never gets the chance to trial new behaviors and receive updated feedback. Second, DPO was fundamentally designed for autoregressive large language models operating in discrete token spaces. The mathematical assumptions underlying the DPO derivation might simply be less robust when applied to the dense, continuous control dynamics required by tasks like Pendulum.

5.4 Sensitivity to Preference Dataset Size

When evaluating performance across varying dataset sizes (K) in Figure 4, both algorithms show a surprising invariance to the volume of preference data.

For PPO-RLHF, the lack of scaling is due to a performance ceiling. Because CartPole and Pendulum are relatively low-dimensional control environments, $K = 500$ preference pairs are already sufficient to learn a directionally accurate reward model. Consequently, providing $10,000$ pairs offers no additional actionable signal; the PPO algorithm is already able to reach maximum return.

For DPO, increasing K also fails to bridge the gap to expert performance, though for different reasons. While DPO reliably learns a policy slightly better than π_2 on CartPole, it plateaus with high variance. This suggests that the bottleneck for DPO in these environments is not a lack of data, but rather an algorithmic constraint. Because DPO strictly optimizes over the state-action distributions present in the dataset, its maximum performance is bounded by the behavioral support of π_1 and π_2. Without the ability to actively explore and generate its own intermediate transitions, DPO cannot stitch together a fully optimal control policy, regardless of how many static preference pairs it observes.

6 Conclusion

In this project, we compared PPO-RLHF and Direct Preference Optimization (DPO) in discrete (CartPole-v1) and continuous (Pendulum-v1) control environments. Our results demonstrate that the traditional, two-stage PPO-RLHF pipeline is highly robust, successfully recovering expert-level performance in both domains. Even with varying reward topologies, the learned proxy models provided sufficient directional signal for PPO to actively explore and converge to optimal policies.

Conversely, DPO struggled to improve meaningfully in CartPole and suffered from severe policy collapse in Pendulum. Its strictly offline formulation restricts it to the state-action distributions of the static preference dataset. Without the ability to actively interact with the environment and correct its trajectory, DPO proved ill-suited for the dynamic, continuous nature of classic control tasks, despite its widespread success in language modeling.

Furthermore, we found that neither algorithm benefited from scaling the preference dataset beyond 500 pairs. PPO-RLHF reached a performance ceiling with minimal data, while DPO's plateau was driven by its algorithmic constraints rather than a lack of data. Ultimately, our findings highlight that while single-stage offline methods like DPO are highly efficient for text alignment, the active environmental exploration inherent to PPO-RLHF remains indispensable for standard reinforcement learning tasks.

7 Code Availability

The source code, training scripts, experiments and poster presentation are available at the following link: https://github.com/mgatti03/EE-568-Reinforcement-Project

References

[1] L. Ouyang *et al.*, “Training language models to follow instructions with human feedback,” 2022, [Online]. Available: https://arxiv.org/abs/2203.02155

[2] P. Christiano, J. Leike, T. B. Brown, M. Martic, S. Legg, and D. Amodei, “Deep reinforcement learning from human preferences,” in *Advances in Neural Information Processing Systems*, 2017.

[3] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.

[4] R. Zheng, S. Dou, S. Gao, and others, “Secrets of RLHF in large language models part I: PPO,” *arXiv preprint arXiv:2307.04964*, 2023.

[5] R. Rafailov, A. Sharma, E. Mitchell, C. D. Manning, S. Ermon, and C. Finn, “Direct preference optimization: Your language model is secretly a reward model,” 2023.

[6] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, “Stable-Baselines3: Reliable Reinforcement Learning Implementations,” *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021, [Online]. Available: https://jmlr.org/papers/v22/20-1364.html

[7] M. Towers *et al.*, “Gymnasium: A Standard Interface for Reinforcement Learning Environments.” [Online]. Available: https://github.com/Farama-Foundation/Gymnasium

[8] G. Brockman *et al.*, “OpenAI Gym,” 2016, [Online]. Available: https://arxiv.org/abs/1606.01540

[9] A. M. Law, *Simulation modeling and analysis*, 5th ed. New York, NY: McGraw-Hill Education, 2015.